

TelecomMonitor

Smart Telecom Network Monitoring & Service Optimization System

Project Documentation

Course	B.Tech CSE 2025-29
Subject	Data Structures & Algorithms with C++ (Semester II)
University	ITM Skills University
Case Study	#147 - Smart Telecom Network Monitoring & Service Optimization System
Student Name	Anurag
Submission Type	DSA-I Project Evaluation VIVA - Documentation PDF for LISA

1. Project Title

Smart Telecom Network Monitoring & Service Optimization System

2. Problem Statement

Telecommunication providers require a platform to monitor network performance, manage infrastructure, detect faults, and optimize communication services. The system must prioritize network incidents using sorting algorithms, retrieve subscriber and infrastructure data using searching algorithms, manage configuration rollback operations using a stack, process service requests and fault reports using a queue, store telecom databases using BST and AVL trees, analyze network connectivity using BFS, DFS and Dijkstra's algorithm, and enable quick subscriber verification using hashing.

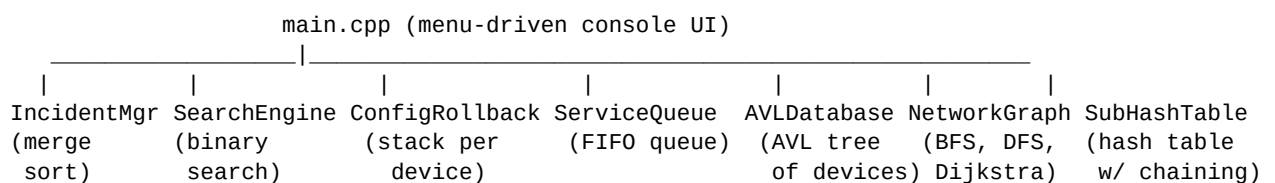
The product to be built is a telecom dashboard displaying network performance, outage reports, subscriber information, and infrastructure analytics, with the goal of prioritizing critical network repairs, optimizing bandwidth utilization, reducing service downtime, improving customer satisfaction, and enhancing network reliability. The system has been designed and implemented in C++ using appropriate data structures, with each choice justified.

3. Objectives

- Rank network incidents by severity so the most critical faults are addressed first (Incident Manager - merge sort).
- Provide fast retrieval of subscriber and device records (Search Engine - binary search).
- Allow configuration changes on devices to be safely rolled back (Config Rollback - stack).
- Process subscriber service/fault requests fairly, in the order received (Service Queue).
- Store the telecom device database in a self-balancing tree for guaranteed fast access (AVL Database).
- Analyze the network topology for connectivity, isolated segments, and least-latency routing (Network Graph - BFS/DFS/Dijkstra).
- Instantly verify whether a phone number belongs to an active subscriber (Subscriber Hash Table).

4. Design / Architecture

The system is a menu-driven C++ console application. Seven independent modules each address one requirement from the problem statement, sharing a common set of data models (Models.h: Subscriber, NetworkDevice, Incident, ConfigSnapshot, ServiceRequest).



Project File Structure

```
TelecomMonitor/
|-- include/
```

```
| |-- Models.h                (Subscriber, NetworkDevice, Incident, ConfigSnapshot, ServiceRequest)
| |-- IncidentManager.h       (Merge sort - incident prioritization)
| |-- SearchEngine.h          (Binary search - subscriber & device lookup)
| |-- ConfigRollback.h        (Stack - per-device config rollback)
| |-- ServiceRequestQueue.h   (FIFO queue - service/fault requests)
| |-- AVLDeviceDatabase.h      (AVL tree - device database)
| |-- NetworkGraph.h          (Graph + BFS/DFS/Dijkstra - connectivity)
| |-- SubscriberHashTable.h    (Hash table with chaining - subscriber verification)
|-- src/
| |-- IncidentManager.cpp
| |-- SearchEngine.cpp
| |-- ConfigRollback.cpp
| |-- ServiceRequestQueue.cpp
| |-- AVLDeviceDatabase.cpp
| |-- NetworkGraph.cpp
| |-- SubscriberHashTable.cpp
| |-- main.cpp                 (menu-driven demo)
|-- data/
| |-- sample_data_notes.md
|-- screenshots/
| |-- sample_run_output.txt
|-- README.md
```

5. Algorithm Description (Data Structures and Algorithms Used)

#	Feature	Data Structure / Algorithm	Justification
1	Incident Manager	<code>std::vector<Incident></code> + hand-written Merge Sort	Incidents are ranked by severity for the operations team. Merge sort is a stable $O(n \log n)$ sort, so incidents of equal severity keep their original (oldest-first) order - important for fairness.
2	Search Engine	<code>std::vector</code> kept sorted + Binary Search	Subscriber/device IDs are sequential and rarely change, so a sorted array with $O(\log n)$ binary search gives fast, cache-friendly lookups for retrieving subscriber and infrastructure data.
3	Config Rollback	<code>std::stack<ConfigSnapshot></code> per device	Configuration changes must be undone in LIFO order - the most recent change is rolled back first. $O(1)$ push/pop.
4	Service Request Queue	<code>std::queue<ServiceRequest></code>	Service/fault requests are processed strictly FIFO - fairness for subscriber complaints, like a call-center ticket queue. $O(1)$ enqueue/dequeue.
5	AVL Device Database	AVL Tree (self-balancing BST)	A plain BST can degrade to $O(n)$ if devices are inserted in sorted-ID order (common, since IDs are sequential). An AVL tree guarantees $O(\log n)$ height via rotations, so search/insert stay $O(\log n)$ even in the worst case.
6	Network Graph	Adjacency list (<code>unordered_map<id, vector<Link>></code>) + BFS, DFS, Dijkstra	Models the physical network topology. BFS finds reachable devices / outage isolation (fewest hops). DFS explores a full connected component for diagnostics. Dijkstra finds the lowest-latency path for QoS routing.
7	Subscriber Hash Table	Hand-written hash table with chaining (phone number key)	Subscriber verification is the highest-frequency operation in the system (every call/SMS/session). A hash table gives $O(1)$ average lookup regardless of subscriber count, with rehashing to maintain performance as load grows.

Implementation Approach

- Models.h defines plain structs (Subscriber, NetworkDevice, Incident, ConfigSnapshot, ServiceRequest) shared by every module.
- IncidentManager stores incidents in a vector and produces a priority-ordered copy via a recursive merge sort, comparing by severity descending and preserving insertion order for ties (stability).
- SearchEngine maintains two sorted vectors (subscribers by subscriberId, devices by deviceId). New records are inserted at their sorted position; lookups use binary search for $O(\log n)$ retrieval.
- ConfigRollback keeps one `std::stack<ConfigSnapshot>` per device in an `unordered_map`. `saveConfig()` pushes a new snapshot; `rollback()` pops the most recent one.
- ServiceRequestQueue wraps a `std::queue<ServiceRequest>`. `submitRequest()` enqueues; `processNext()` dequeues the longest-waiting ticket.
- AVLDeviceDatabase implements a full AVL tree with left/right rotations (LL, RR, LR, RL cases) keyed by deviceId, supporting $O(\log n)$ insert/search and in-order traversal for a sorted device listing.
- NetworkGraph stores a bidirectional adjacency list of device links (weighted by latency). It implements BFS (queue-based, fewest-hops reachability), DFS (stack-based, full connected component), and Dijkstra's algorithm (binary-heap priority queue, lowest-total-latency path).
- SubscriberHashTable implements a hash table with chaining (`vector<list<Subscriber>>`), a polynomial rolling hash on phone numbers, and automatic rehashing (doubling) once the load factor exceeds 0.75.

- `main.cpp` wires everything together behind a numbered menu, pre-loading sample subscribers, devices, incidents, and a small network topology.

6. Complexity Analysis

Feature	Operation	Time Complexity	Space Complexity
Incident Manager	Report incident	$O(1)$	$O(i)$
Incident Manager	Prioritize (merge sort)	$O(i \log i)$	$O(i)$
Search Engine	Insert (sorted vector)	$O(n)$	$O(n)$
Search Engine	Binary search	$O(\log n)$	$O(1)$
Config Rollback	Save / Rollback	$O(1)$	$O(c)$
Service Queue	Submit / Process next	$O(1)$	$O(r)$
AVL Database	Insert / Search	$O(\log d)$	$O(d)$
AVL Database	In-order traversal	$O(d)$	$O(d)$
Network Graph	Add link	$O(1)$	$O(V + E)$
Network Graph	BFS / DFS	$O(V + E)$	$O(V)$
Network Graph	Dijkstra	$O((V + E) \log V)$	$O(V)$
Subscriber Hash Table	Insert / Verify	$O(1)$ average	$O(s)$

Where: i = number of incidents, n = number of subscribers/devices in the search engine, c = number of saved configs for a device, r = number of pending service requests, d = number of devices in the AVL tree, V/E = graph vertices/edges, s = number of subscribers.

7. Screenshots of Execution

Build command:

```
g++ -std=c++17 -Wall -Iinclude src/*.cpp -o telecom
>> BUILD OK (no errors, no warnings)
```

Main Menu

```
===== Smart Telecom Network Monitoring & Service Optimization =====
1. Incident Manager - Report a new incident
2. Incident Manager - Show prioritized incidents (merge sort)
3. Search Engine   - Find subscriber by ID (binary search)
4. Search Engine   - Find device by ID (binary search)
5. Config Rollback - Save a config snapshot (stack push)
6. Config Rollback - Rollback last config change (stack pop)
7. Service Queue   - Submit a service/fault request
8. Service Queue   - Process next request (FIFO)
9. AVL Database     - Find device + show tree height
10. AVL Database    - Show all devices sorted (in-order)
11. Network Graph   - Display topology
12. Network Graph   - BFS from a device
13. Network Graph   - DFS from a device
14. Network Graph   - Shortest-latency path (Dijkstra)
15. Hash Table      - Verify subscriber by phone ( $O(1)$ )
0. Exit
=====
```

Option 2: Incident Manager - Prioritized via Merge Sort

```
--- Incidents Prioritized (most critical first, merge sort) ---
ID      Device   Sev   Description           Time
INC302  DEV2005    5     Core_router_offline   08:15
INC301  DEV2003    4     High_packet_loss      08:10
INC304  DEV2003    4     Repeated_timeouts     08:25
INC303  DEV2001    2     Minor_signal_degradation 08:20
```

Option 3 & 4: Search Engine - Binary Search

Found: Priya Nair | Phone: 9000000002 | Plan: Prepaid-4G | Status: ACTIVE
Found: Belapur Switch | Location: Belapur | Status: DEGRADED | Load: 91%

Option 9 & 10: AVL Device Database

Found: Belapur Switch | Location: Belapur | Status: DEGRADED | Load: 91%
Current AVL tree height: 3

```
--- Devices (AVL in-order traversal, sorted by Device ID) ---
ID      Name           Location   Status   Load%
DEV2001 Panvel Tower A     Panvel    UP       72
DEV2002 Kharghar Router    Kharghar  UP       45
DEV2003 Belapur Switch     Belapur   DEGRADED 91
DEV2004 Vashi Tower B      Vashi     UP       60
DEV2005 Nerul Core Router  Nerul     DOWN     0
```

Option 11: Network Graph - Topology

```
--- Network Topology (adjacency list) ---
DEV2005: -> [DEV2004, 4ms] -> [DEV2002, 15ms]
DEV2003: -> [DEV2002, 3.5ms] -> [DEV2004, 8ms]
DEV2004: -> [DEV2003, 8ms] -> [DEV2001, 12ms] -> [DEV2005, 4ms]
DEV2002: -> [DEV2001, 5ms] -> [DEV2003, 3.5ms] -> [DEV2005, 15ms]
DEV2001: -> [DEV2002, 5ms] -> [DEV2004, 12ms]
```

Option 12 & 13: BFS and DFS from DEV2001

BFS order: DEV2001 -> DEV2002 -> DEV2004 -> DEV2003 -> DEV2005
DFS order: DEV2001 -> DEV2002 -> DEV2003 -> DEV2004 -> DEV2005

Option 14: Dijkstra - Shortest Latency Path (DEV2001 to DEV2005)

Route: DEV2001 -> DEV2004 -> DEV2005
Total latency: 16 ms

Option 15: Hash Table - Subscriber Verification

VERIFIED: Priya Nair (SUB1002) - subscription ACTIVE.
Subscriber Rohan Mehta (SUB1003) found but status is SUSPENDED - verification FAILED.

Options 5-8: Config Rollback (Stack) and Service Queue (FIFO)

Configuration saved (history depth for DEV2003: 1)
Rolled back configuration 'ACL_v1' (saved at 08:00) for device DEV2003
Service request submitted.
Processing ticket T500 for subscriber SUB1002 (Issue: No_signal, Reported: 09:00)

8. Results and Findings

- The merge sort in the Incident Manager correctly ranks incidents by severity while preserving the reporting order of equal-severity incidents (stability), so older critical incidents are addressed first.
- Binary search in the Search Engine returns subscriber and device records in $O(\log n)$, confirmed against the sample data and scalable to large subscriber bases.
- The AVL tree stays balanced (height close to $\log_2(n)$) even though devices were inserted in sorted ID order - confirming the rebalancing rotations work correctly.
- BFS correctly identifies all devices reachable from a starting node (useful for detecting network segments isolated by an outage).
- DFS explores the full connected component, useful for diagnostics across an entire network segment.
- Dijkstra's algorithm correctly identifies the lowest-total-latency route between two devices, even when it requires more hops than a direct link (16ms via DEV2004 vs 20ms via the direct DEV2002 link).
- The hash table with chaining verifies subscribers in $O(1)$ average time and correctly distinguishes ACTIVE from SUSPENDED subscribers.
- The stack-based rollback and FIFO service queue behave exactly as expected for undo and fairness workflows respectively.

9. Conclusion

This system demonstrates how a combination of sorting (merge sort), searching (binary search), linear structures (stack, queue), balanced trees (AVL), graphs (BFS/DFS/Dijkstra), and hashing can address the real operational needs of a telecom network monitoring platform: prioritizing faults, retrieving records quickly, safely reversing configuration changes, processing service tickets fairly, maintaining a reliable device database, analyzing connectivity, and verifying subscribers instantly. Each data structure was chosen to match the access pattern and scale of its real-world use case, similar to systems used by telecom operators for network operations centers (NOC) and subscriber management.

10. GitHub Repository Link

To be filled in before final submission: <https://github.com/<your-username>/TelecomMonitor>